

FROM 550+ REAL MENTORSHIP SESSIONS

The 50 SDET Interview Questions That Keep Showing Up

Not a random list from the internet. These are the questions my mentees actually get asked at Amazon, Microsoft, Google, Adobe, Salesforce and top product companies, with a note on what the interviewer is really checking in each one.

Sahil Kapoor

Software Engineer at Amazon. I build CI/CD platforms, device infrastructure and AI agents, and I have sat on both sides of the interview table more times than I can count.

topmate.io/sahil_kapoor

Before you start

I have run more than 550 mentorship sessions. Different companies, different levels, different people. The surprising part is how small the real question pool is. The same fifty questions, in some phrasing or another, decide most SDET interviews.

Two ways to use this document. The wrong way: read it once, feel prepared, move on. The right way: for each question, close the document and answer out loud before reading my note. If your answer misses what the note says the interviewer is checking, that question goes on your revision list.

The notes are deliberately short. They tell you what a strong answer contains, not a script to memorize. Interviewers can smell a memorized answer within seconds, and it costs you more than an honest "I have not worked with that."

Part 1. Testing fundamentals

1. What is the difference between smoke, sanity and regression testing, and where does each sit in a release cycle?

Checking: whether you have actually run a release, or just read definitions. Strong answers place each one at a moment in time: smoke after every deploy, sanity on a fresh build before deeper testing, regression before release. Bonus if you mention what you do when there is no time to run all of them.

2. How do you decide what NOT to automate?

Checking: judgment. Weak candidates say "automate everything." Strong ones talk about cost per run versus frequency of change, one-time flows, exploratory testing, and UI areas changing weekly where automation becomes maintenance debt.

3. A test passes on your machine and fails in CI. Walk me through your debugging order.

Checking: a method, not a guess. Strong shape: reproduce with CI's exact inputs first, then diff the environments (versions, data, timing, parallelism, network), then look at the failure artifacts before touching code. Naming a real cause you once found (a timezone, a race, a stale dependency) makes this answer land.

4. What makes a test flaky? Name causes you have personally fixed.

Checking: scars. Timing waits, order dependence, shared test data, animation and rendering races, retries hiding real bugs. The word "personally" is the whole question. Have two true stories ready.

5. Your suite is green every day, yet bugs still escape to production. What is wrong?

Checking: whether you confuse test activity with test value. Strong answers question the suite itself: coverage concentrated on stable paths, assertions too shallow, tests that skip silently, environments that differ from production. The best candidates mention measuring escaped defects, not pass rate.

6. How do you measure the quality of a test suite?

Checking: that you look beyond coverage percentage. Escaped-defect rate, time to detect, flake rate, how often the suite catches a real regression pre-merge. Coverage tells you what ran, not what was verified.

7. You have two hours before release and four hundred tests. What do you run?

Checking: prioritization under pressure. Risk-based selection: what changed in this release, the PO customer journeys, the areas with recent defect history. Saying "I would ask for more time" is a worse answer than showing how you cut.

Part 2. Automation and frameworks

8. Explain the page object model. When does it break down?

Checking: both halves. Everyone knows the first (locators and actions live with the page, tests read like intent). Few answer the second: deep component reuse across pages, dynamic UIs, over-abstracted base classes nobody understands. Knowing the limits proves you have lived with one at scale.

9. Explicit versus implicit waits. And why is a hard sleep in a test considered a serious smell?

Checking: understanding of why suites get slow and flaky. A sleep is always either too long (wasted minutes across thousands of runs) or too short (flake). Waiting on a condition is waiting on truth; sleeping is waiting on hope.

10. The element you need has no id and no stable attributes. What do you do?

Checking: practical seniority. Relative locators from a stable anchor, text with care about localization, accessibility attributes, and the senior move: go ask the developers to add a test id, because locator archaeology is a symptom, not a job.

11. How would you make a 4-hour suite finish in 15 minutes?

Checking: whether you think in infrastructure. Parallel workers, test sharding, isolating the slow tail, killing redundant tests, moving checks down the pyramid to API and unit level. Numbers help: say what you parallelized and what it saved in a real project.

12. What is the test pyramid, and where do most teams get it wrong?

Checking: honesty about the industry's favorite diagram. Most teams run an inverted pyramid, hundreds of slow UI tests over a thin unit layer, because UI tests are easy to demo. Strong answers explain the cost curve: feedback speed and debuggability fall off a cliff as you go up the stack.

13. How do you manage test data so that a thousand runs a day do not corrupt each other?

Checking: scale experience. Per-run data creation and teardown, isolated accounts per worker, factories over fixtures, and what happens when a shared account gets wedged into a bad state. If you have a war story about shared state, tell it.

14. How do you keep tests independent, and why does order dependence keep creeping in?

Checking: root-cause thinking. It creeps in because state leaks are invisible while tests happen to run in a lucky order. Each test builds its own state and cleans up; run the suite shuffled in CI so order bugs surface early instead of on release day.

15. Selenium, Playwright, Cypress, Appium. How do you actually choose?

Checking: a decision process, not a fan opinion. Platform targets, language of the team, ecosystem maturity, parallelism model, who will maintain it in two years. The honest answer includes: the framework matters less than the discipline of the people writing the tests.

16. Describe a framework you built or significantly extended. What would you change today?

Checking: ownership plus self-awareness. The second half is the differentiator. Everyone praises their own framework; strong candidates name a design decision they regret and why. Have one ready and be specific.

17. A known-flaky test sits in your merge gate. Retry it, quarantine it, or delete it? Defend your choice.

Checking: whether you understand trust. Blind retries train everyone to ignore red. A good answer: quarantine with an owner and an expiry, fix or delete by the deadline, and never let a bare retry hide a real intermittent bug. If you have seen a "flaky" test that was actually a real production bug, that story wins the round.

Part 3. Coding, the SDET round

18. String manipulation live: reverse words in a sentence, in place.

Checking: fluency without a library crutch. The classic trick is reverse the whole string, then reverse each word. Talk while you code; silence is the real failure mode in these rounds.

19. Find duplicates in a large dataset. Follow-up: the data does not fit in memory.

Checking: whether you scale an answer. Hash set first, then for the follow-up: chunking, sorting on disk, or hashing to buckets. The follow-up is the actual question.

20. Parse a log file and return the top five most frequent error codes.

Checking: the most SDET-flavored coding question there is. Dictionary count plus a heap for top-K. Mention how you would handle a malformed line, because real logs always have one.

21. Write a retry helper with exponential backoff and a maximum attempt count.

Checking: production instincts in a ten-line function. Backoff with jitter, retry only on retryable errors, give up loudly. Bonus points for saying where you would NOT use it: around assertions, where retries mask bugs.

22. Implement a simple rate limiter class.

Checking: a design-flavored coding question that separates levels. Token bucket is the expected shape: capacity, refill rate, a clean allow method. Being able to say why token bucket over a fixed window (burst handling) moves you up a level.

23. Two Sum and its variants.

Checking: hash map reflexes. It still opens more coding rounds than any other question. If you cannot do this in five minutes cold, your prep order is wrong.

24. Valid parentheses, and bracket matching in general.

Checking: stack fundamentals. Frequently extended into "now validate this JSON-like structure," which is exactly the kind of parsing SDETs write for real.

25. Given two JSON API responses, write a deep diff that reports every differing path.

Checking: recursion plus the day job. Handle nested objects, arrays, type mismatches, missing keys. This is the question where an SDET can genuinely outshine a developer, because you have done it for real.

Part 4. API testing

26. The endpoint returns 200. What else do you assert before calling it tested?

Checking: depth beyond the status code. Schema and types, business values, headers, latency budget, side effects actually persisted, error body shape on the negative path. "200 is where testing starts, not ends" is the right posture.

27. What is idempotency, which methods should have it, and how do you test it?

Checking: a concept that separates API testers from API callers. Send the same request twice, assert the state changed once. Mention idempotency keys on POST for payment-class operations and you are in the top tier of answers.

28. How do you test authorization, not just authentication?

Checking: that you know they are different bugs. Valid token but wrong role, accessing another user's resource by swapping ids, expired versus malformed versus missing token. The horizontal-access case (user A reads user B's data) is the one that becomes a security incident.

29. Contract testing versus end-to-end testing. When is each the right tool?

Checking: whether you can protect service boundaries cheaply. Contracts catch breaking schema changes per integration without spinning up the world; E2E validates the few journeys where everything must truly connect. The wrong answer is treating them as competitors.

30. What is in your negative-testing checklist for a POST endpoint?

Checking: systematic destruction. Missing fields, wrong types, boundary sizes, duplicate submission, malformed encoding, injection strings, oversized payloads, wrong content type. Reciting eight of these fluently signals you have a checklist in your head, which is the point.

31. How do you test rate limiting?

Checking: precision. Hit the limit exactly, assert the 429 and its retry headers, assert recovery after the window, and test that limits are per-key rather than global. Sloppy answers just say "send lots of requests."

32. The API adds new response fields every sprint. How do your tests survive?

Checking: maintainability philosophy. Assert what you own strictly, tolerate additive change, fail loudly on removed or retyped fields. A test suite that breaks on every harmless addition trains the team to stop trusting it.

Part 5. CI/CD and infrastructure

33. Design the quality gates for a CI/CD pipeline. What runs pre-merge, post-merge, pre-production?

Checking: that you place tests by feedback speed and blast radius. Fast and deterministic pre-merge; integration and slower suites post-merge; smoke plus progressive rollout checks pre-production. Every gate must be trusted, because one flaky gate poisons the whole pipeline's credibility.

34. Deployment passed every test and still broke production. Run the post-mortem.

Checking: systems thinking without blame. Config differences, data shape differences, traffic patterns tests never produce, feature flags in different states, monitoring gaps. Strong answers end with which layer gets a new check so this class of failure cannot recur.

35. Pipeline time keeps growing as the suite grows. How do you keep it under fifteen minutes?

Checking: whether you treat pipeline speed as a product. Parallelization, selecting tests affected by the change, pushing checks down the pyramid, caching, and deleting tests that have never failed for a real reason. That last one takes organizational courage; mentioning it shows you have the spine.

36. Canary versus blue-green deployment. How does your test strategy differ between them?

Checking: release engineering literacy. Canary shifts weight onto monitoring and automated rollback signals on the small slice; blue-green needs a solid smoke pass on the idle environment before the switch. The test suite is only half the answer; the other half is the metrics that decide rollback.

37. Where do your test environments get their data?

Checking: realism versus safety. Synthetic data is safe but misses real-world weirdness; production copies are realistic but a privacy problem. Strong answers: seeded synthetic baselines plus anonymized samples for shape, and a hard line on never using raw customer data.

38. How do you test infrastructure-as-code changes before they hit production?

Checking: that quality thinking extends below the application. Plan diffs reviewed like code, policy checks, deploying to a staging account first, and a rollback path. If you have a story of an infra change that deleted something it should not have, this is where it earns its keep.

Part 6. Test design at scale

39. Design a test automation strategy for a brand-new product, from zero.

Checking: the classic SDET design round. Do not start with a tool. Start with risk: what are the P0 user journeys, what breaks the business. Then layers (unit, API, UI, performance), then infrastructure (CI hooks, environments, data, reporting), then the rollout order. Tool names come last, and it should be obvious you could swap them.

40. Design a real-device testing lab, or a cross-browser grid.

Checking: physical-world experience. Device allocation and queuing, session cleanup between runs, health monitoring for devices that wedge, video and log artifacts, and what happens when a device is claimed by two runs at once. This question rewards people who have actually operated one.

41. Design the reporting system for ten thousand test runs a day.

Checking: data thinking. Results store, deduplication, flake detection across history, trend dashboards per team, and alerting that separates "new failure" from "known failure." The killer feature to mention: history-aware verdicts, because a failure that has happened 400 times is information, not news.

42. The product integrates with a thousand external services. Design the testing approach.

Checking: that you do not answer "E2E tests for everything." Contract tests per integration against recorded or sandboxed responses, continuous verification that detects provider drift before customers do, a thin E2E layer for the flows that matter most, and monitoring as the final tester.

43. Design load testing for a live event where a million users arrive in two minutes.

Checking: the thundering-herd scenario. Ramp profiles that match reality rather than gentle linear ramps, testing the queue and backpressure behavior, knowing your autoscaling reaction time versus the spike speed, and pre-scaling decisions. The insight interviewers want: autoscaling that takes ten minutes cannot save a two-minute spike.

44. How do you test a zero-downtime migration?

Checking: care under irreversibility. Dual-write or shadow-read verification, data reconciliation counts, a rollback plan tested in advance, and canarying the cutover. Saying "verify row counts match" is table stakes; verifying content checksums and behavior parity is the real answer.

Part 7. Testing in the AI era

45. The feature's output is generated by a model and different every run. How do you test it?

Checking: the new frontier, and most candidates have nothing. Strong shape: exact-match assertions are dead, so you assert properties (format, constraints, required facts present), score outputs against intent, and measure success rate across many runs instead of pass or fail on one. This question is rapidly becoming a differentiator between candidates.

46. What is LLM-as-judge, and what are its failure modes?

Checking: depth beyond the buzzword. A second model grades outputs against a rubric or ground truth. Failure modes worth naming: the judge drifts, the judge shares biases with the model it grades, and judges are non-deterministic too. Controls: fixed rubrics, spot-checking judge decisions against human labels, tracking agreement over time.

47. How do you build an evaluation set for an AI feature?

Checking: whether you can make quality measurable. Real cases from production, hard cases and edge cases deliberately included, ground-truth labels, and the rule that turns it into an engine: every real failure found in production becomes a permanent eval case. It is a regression suite that grows forever.

48. An AI agent picks the right tool 95 percent of the time. How would you catch a silent regression to 90?

Checking: statistical instincts. A single test cannot see this. You need a scored eval set run on every change, a baseline, and a gate on the delta. Bonus: mention that the dangerous failures are silent, an agent confidently doing the wrong thing raises no exception, so behavior must be measured, not just errors counted.

49. How do you use AI tools in your own testing work?

Checking: honesty and ownership. Good answer shape: what you use them for (generating test scaffolding, exploring unfamiliar code, drafting cases), how you verify everything before it ships, and one concrete example. The failing answer is either "I do not use them" or "I use whatever it generates." You own every line either way.

50. Will AI replace testers?

Checking: composure on the trap question. The strong answer is neither defensive nor dismissive: AI is automating test execution and generation, while creating an entirely new class of hard verification problems, since probabilistic systems need engineers who can measure behavior instead of asserting outputs. The skill was never clicking buttons; it was proving software works. That skill now has a bigger target.

What to do with this document

Do not read it twice. Work it. Out loud, one part per sitting, marking every question where your answer missed the note. Those marks are your prep plan.

If you want to pressure-test your answers against someone who has heard thousands of them, a mock interview with real-time feedback is the fastest correction there is. That is what I do.

topmate.io/sahil_kapoor: mock interviews, prep plans, resume reviews, and a 30-day mentorship program for engineers who are serious about the move.

